



тема

Структуры хранения информации о файле

Цель

Научиться на практических примерах работать со структурами хранения информации о файле

Задачи

Научиться работать со структурами:

- FILE,
- struct stat,
- файловые дескрипторы,
- буферы и связанными с ними механизмы

Часть 1.

Структура FILE (примеры 1–7)

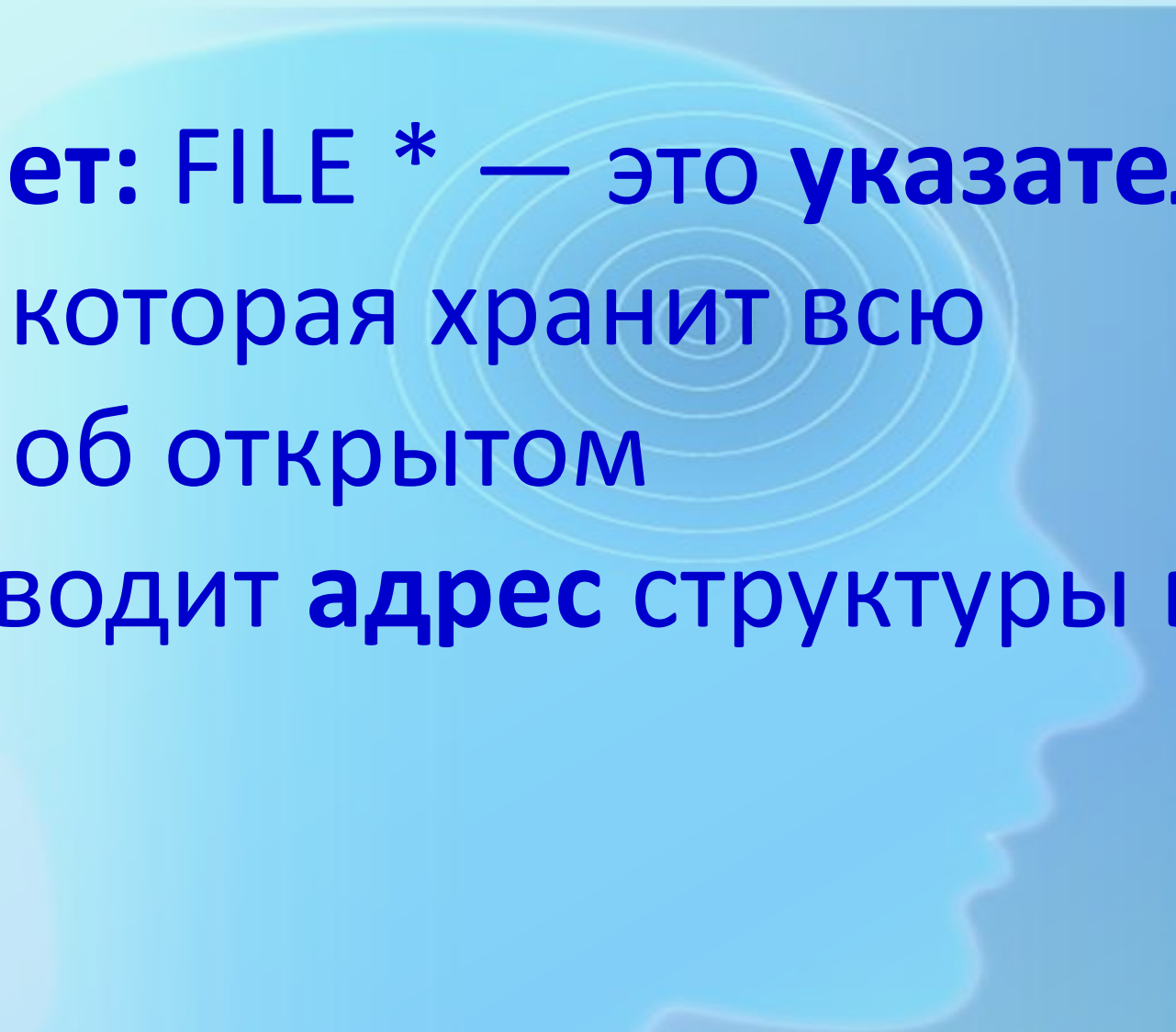


Пример 1. Что такое FILE * — объявление и проверка

```
#include <stdio.h>
```

```
int main(void) {  
    FILE *fp = fopen("hello.txt", "r");  
    if (fp == NULL) {  
        perror("fopen");  
        return 1;  
    }  
    printf("Файл открыт, указатель: %p\n", (void *)fp);  
    fclose(fp);  
    return 0;  
}
```





Что показывает: FILE * — это указатель на структуру, которая хранит всю информацию об открытом файле. %p выводит адрес структуры в памяти.

Пример 2. Размер структуры FILE

```
#include <stdio.h>
```

```
int main(void) {  
    printf("sizeof(FILE) = %zu байт\n", sizeof(FILE));  
    return 0;  
}
```

Что показывает: структура FILE занимает **фиксированный объём памяти**. На разных системах — **разный**, но обычно **от 100 до 500 байт**.

Пример 3. Позиция в файле через ftell

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("hello.txt", "r");
    if (!fp) return 1;

    int c;
    while ((c = fgetc(fp)) != EOF) {
        long pos = ftell(fp);
        printf("Символ '%c' на позиции %ld\n", c, pos);
    }

    fclose(fp);
    return 0;
}
```

Что показывает: внутри FILE хранится **текущая позиция** в файле. ftell её возвращает.

Пример 4. Флаги состояния: feof и ferror

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("hello.txt", "r");
    if (!fp) return 1;

    while (fgetc(fp) != EOF) { }

    if (feof(fp)) printf("Достигнут конец файла\n");
    if (ferror(fp)) printf("Произошла ошибка\n");

    fclose(fp);
    return 0;
}
```

Что показывает: в FILE есть **флаги состояния** — EOF и error. Их проверяют через feof и ferror.

Пример 5. Сброс буфера через fflush

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("out.txt", "w");
    if (!fp) return 1;

    fputs("Данные в буфере", fp);
    fflush(fp); // сброс буфера на диск
    printf("Буфер сброшен\n");

    fclose(fp);
    return 0;
}
```

Что показывает: FILE содержит **буфер**. fflush принудительно записывает его содержимое на диск.

Пример 6. Буферизация: setvbuf

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("out.txt", "w");
    if (!fp) return 1;

    char buf[1024];
    setvbuf(fp, buf, _IOFBF, sizeof(buf)); // полная буферизация

    fputs("Данные", fp);
    fclose(fp);
    return 0;
}
```

Что показывает: можно **управлять буфером** FILE через setvbuf. Режимы:

_IOFBF — полная буферизация.

_IOLBF — построчная.

_IONBF — без буферизации.

Пример 7. Временный файл через tmpfile

```
#include <stdio.h>

int main(void) {
    FILE *fp = tmpfile();
    if (!fp) return 1;

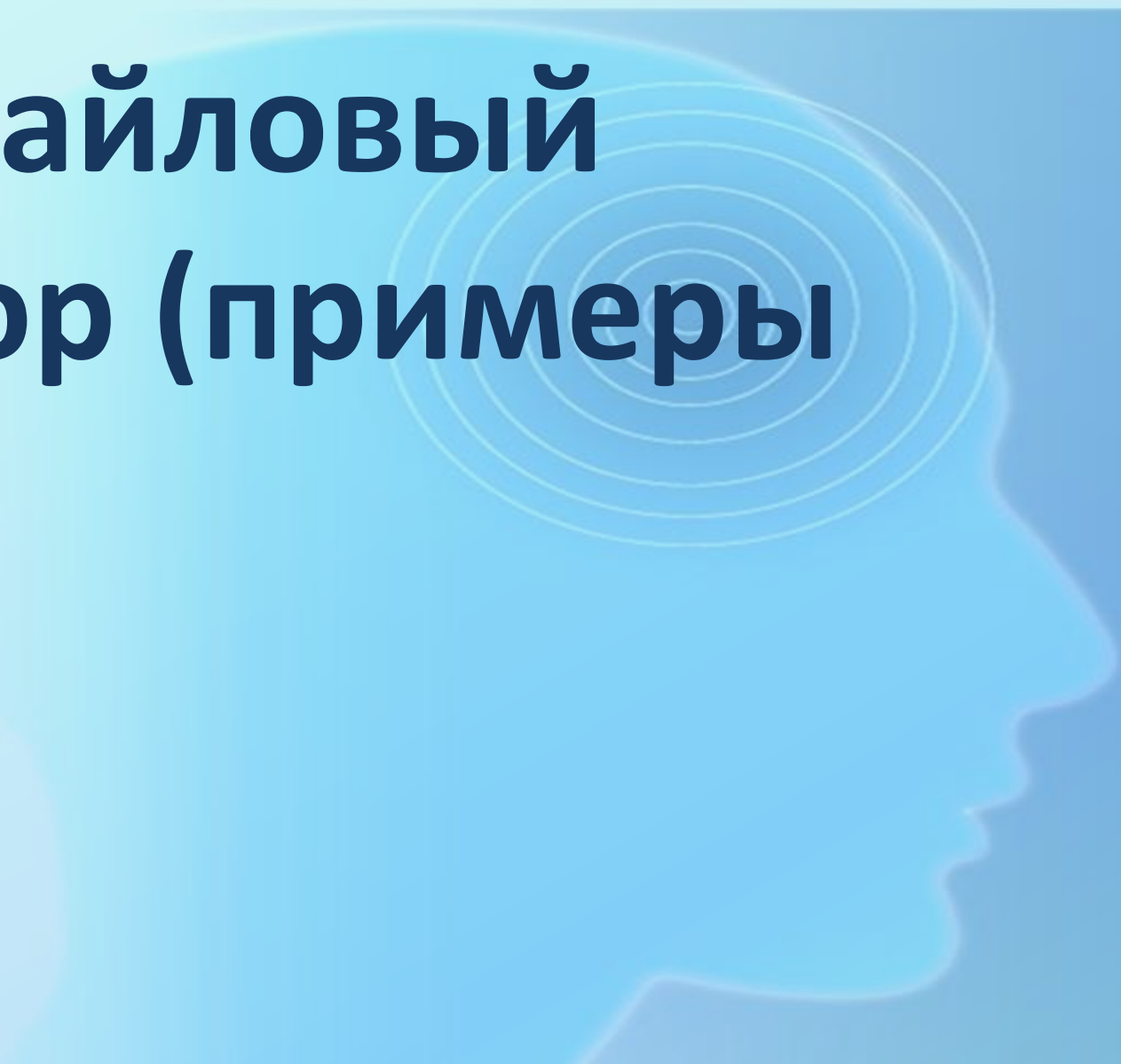
    fputs("Временные данные", fp);
    rewind(fp);

    char buf[100];
    fgets(buf, sizeof(buf), fp);
    printf("Прочитано: %s\n", buf);

    fclose(fp); // файл автоматически удаляется
    return 0;
}
```

Что показывает: tmpfile создаёт **временный файл**, который автоматически удаляется при закрытии. Структура FILE хранит его дескриптор.

Часть 2. Файловый дескриптор (примеры 8–12)



Пример 8. Получение дескриптора из FILE через fileno

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("hello.txt", "r");
    if (!fp) return 1;

    int fd = fileno(fp);
    printf("Файловый дескриптор: %d\n", fd);

    fclose(fp);
    return 0;
}
```

Что показывает: внутри FILE хранится **файловый дескриптор** (int). fileno его возвращает.

Пример 9. Стандартные дескрипторы

```
#include <stdio.h>
```

```
int main(void) {  
    printf("stdin: fd = %d\n", fileno(stdin));  
    printf("stdout: fd = %d\n", fileno(stdout));  
    printf("stderr: fd = %d\n", fileno(stderr));  
    return 0;  
}
```

Что показывает: по умолчанию открыты **три дескриптора**:

- 0 — stdin,
- 1 — stdout,
- 2 — stderr.

Пример 10. Дескриптор через open (системный вызов)

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd = open("hello.txt", O_RDONLY);
    if (fd == -1) {
        perror("open");
        return 1;
    }
    printf("Дескриптор: %d\n", fd);
    close(fd);
    return 0;
}
```

Что показывает: системный вызов `open` возвращает **дескриптор** — целое число, индекс в таблице открытых файлов процесса.

Пример 11. Обёртка FILE вокруг дескриптора через fdopen

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd = open("hello.txt", O_RDONLY);
    if (fd == -1) return 1;

    FILE *fp = fdopen(fd, "r");
    if (!fp) return 1;

    int c;
    while ((c = fgetc(fp)) != EOF) putchar(c);

    fclose(fp); // закрывает и fp, и fd
    return 0;
}
```

Что показывает: можно **обернуть** низкоуровневый дескриптор в высокоуровневую структуру FILE через fdopen.

Пример 12. Дублирование дескриптора через dup

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd1 = open("hello.txt", O_RDONLY);
    if (fd1 == -1) return 1;

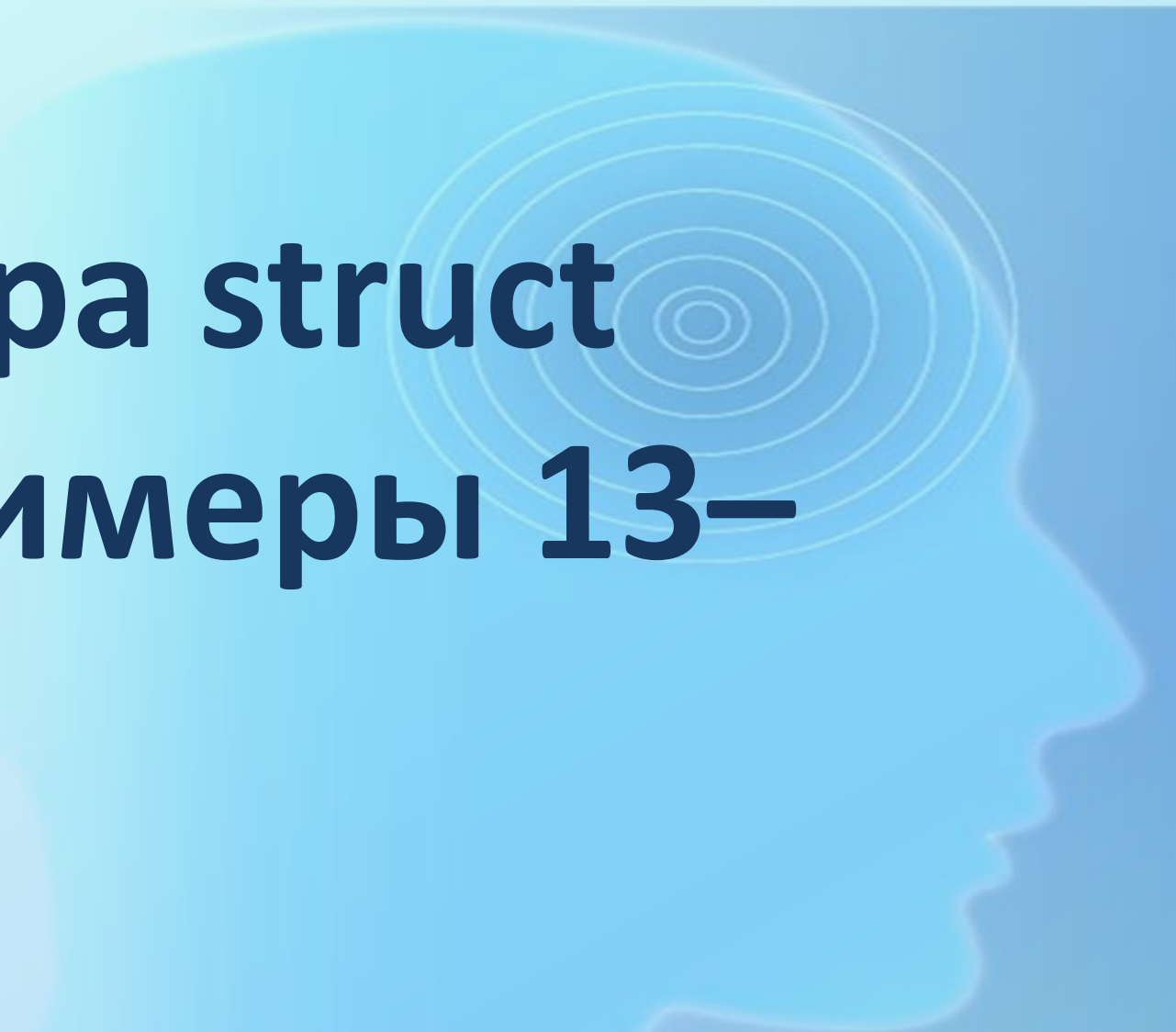
    int fd2 = dup(fd1);
    printf("fd1 = %d, fd2 = %d\n", fd1, fd2);

    close(fd1);
    close(fd2);
    return 0;
}
```

Что показывает: dup создаёт **копию дескриптора**, указывающую на тот же файл. Оба дескриптора независимы, но разделяют позицию.

Часть 3.

**Структура `struct`
`stat` (примеры 13–
17)**



Пример 13. Получение метаданных через stat

```
#include <stdio.h>
#include <sys/stat.h>

int main(void) {
    struct stat st;
    if (stat("hello.txt", &st) == -1) {
        perror("stat");
        return 1;
    }

    printf("Размер: %ld байт\n", (long)st.st_size);
    printf("Inode: %ld\n", (long)st.st_ino);
    printf("Ссылка: %ld\n", (long)st.st_nlink);
    return 0;
}
```

Что показывает: структура struct stat хранит **метаданные файла**: размер, inode, число ссылок.

Пример 14. Тип файла через st_mode

```
#include <stdio.h>
#include <sys/stat.h>

int main(void) {
    struct stat st;
    if (stat("hello.txt", &st) == -1) return 1;

    printf("Тип: ");
    if (S_ISREG(st.st_mode))    printf("обычный файл\n");
    else if (S_ISDIR(st.st_mode)) printf("каталог\n");
    else if (S_ISLNK(st.st_mode)) printf("символьная ссылка\n");
    else                        printf("другое\n");

    return 0;
}
```

Что показывает: поле st_mode хранит **тип файла**. Проверяется макросами S_ISREG, S_ISDIR, S_ISLNK.

Пример 15. Права доступа через st_mode

```
#include <stdio.h>
#include <sys/stat.h>

int main(void) {
    struct stat st;
    if (stat("hello.txt", &st) == -1) return 1;

    printf("Права: ");
    printf("%c", (st.st_mode & S_IRUSR) ? 'r' : '-');
    printf("%c", (st.st_mode & S_IWUSR) ? 'w' : '-');
    printf("%c", (st.st_mode & S_IXUSR) ? 'x' : '-');
    printf("%c", (st.st_mode & S_IRGRP) ? 'r' : '-');
    printf("%c", (st.st_mode & S_IWGRP) ? 'w' : '-');
    printf("%c", (st.st_mode & S_IXGRP) ? 'x' : '-');
    printf("%c", (st.st_mode & S_IROTH) ? 'r' : '-');
    printf("%c", (st.st_mode & S_IWOTH) ? 'w' : '-');
    printf("%c\n", (st.st_mode & S_IXOTH) ? 'x' : '-');

    return 0;
}
```

Что показывает: права доступа **упакованы** в st_mode — три группы по три бита (владелец, группа, остальные).

Пример 16. Временные метки файла

```
#include <stdio.h>
#include <sys/stat.h>
#include <time.h>

int main(void) {
    struct stat st;
    if (stat("hello.txt", &st) == -1) return 1;

    printf("Последний доступ: %s", ctime(&st.st_atime));
    printf("Последнее изменение: %s", ctime(&st.st_mtime));
    printf("Изменение inode:  %s", ctime(&st.st_ctime));
    return 0;
}
```

Что показывает: struct stat хранит **три времени**:

st_atime — доступ,

st_mtime — изменение содержимого,

st_ctime — изменение inode.

Пример 17. fstat — метаданные по дескриптору

```
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd = open("hello.txt", O_RDONLY);
    if (fd == -1) return 1;

    struct stat st;
    if (fstat(fd, &st) == -1) return 1;

    printf("Размер: %ld\n", (long)st.st_size);
    close(fd);
    return 0;
}
```

Что показывает: fstat работает **по дескриптору**, а не по имени файла. Полезно, когда файл уже открыт.

Часть 4. Специальные структуры (примеры 18–20)



Пример 18. Структура DIR для каталогов

```
#include <stdio.h>
#include <dirent.h>
```

```
int main(void) {
    DIR *dir = opendir(".");
    if (!dir) return 1;

    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL)
        printf("%s\n", entry->d_name);

    closedir(dir);
    return 0;
}
```

Что показывает: для каталогов используется **своя структура** — DIR и struct dirent. Поле d_name хранит имя файла.

Пример 19. Структура passwd — информация о пользователе

```
#include <stdio.h>
#include <pwd.h>

int main(void) {
    struct passwd *pw = getpwuid(getuid());
    if (!pw) return 1;

    printf("Пользователь: %s\n", pw->pw_name);
    printf("UID: %d\n", pw->pw_uid);
    printf("Домашний каталог: %s\n", pw->pw_dir);
    return 0;
}
```

Что показывает: в системе есть **другие структуры** для информации о пользователях, группах, устройствах. struct passwd — одна из них.

Пример 20. Сводная информация о файле

```
#include <stdio.h>
#include <sys/stat.h>

int main(void) {
    struct stat st;
    if (stat("hello.txt", &st) == -1) return 1;

    printf("=== Информация о файле ===\n");
    printf("Размер:    %ld байт\n", (long)st.st_size);
    printf("Inode:    %ld\n", (long)st.st_ino);
    printf("Устройство: %ld\n", (long)st.st_dev);
    printf("Ссылка:    %ld\n", (long)st.st_nlink);
    printf("UID:      %d\n", st.st_uid);
    printf("GID:      %d\n", st.st_gid);
    printf("Блоков:    %ld\n", (long)st.st_blocks);
    printf("Размер блока: %ld\n", (long)st.st_blksize);

    return 0;
}
```

Что показывает: все основные поля struct stat в одном примере — **полный «паспорт» файла.**

Сводная таблица: структуры и их назначение

Структура	Где определена	Что хранит
<code>FILE</code>	<code><stdio.h></code>	Буфер, дескриптор, позиция, флаги
<code>struct stat</code>	<code><sys/stat.h></code>	Метаданные: размер, тип, права, время
<code>int fd</code>	—	Файловый дескриптор
<code>DIR</code>	<code><dirent.h></code>	Информация о каталоге
<code>struct dirent</code>	<code><dirent.h></code>	Имя файла, inode
<code>struct passwd</code>	<code><pwd.h></code>	Информация о пользователе
<code>struct tm</code>	<code><time.h></code>	Разложенное время

Что важно запомнить

FILE — непрозрачная структура. Её поля нельзя читать напрямую — только через функции.

struct stat — основной источник метаданных файла.

Дескриптор (int fd) — низкоуровневый «пропуск» к файлу.

fileno связывает FILE и дескриптор.

fdopen делает обратное — оборачивает дескриптор в FILE.

Итог

20 примеров покрывают:

- **FILE *** — высокоуровневая структура,
- **файловые дескрипторы** — низкоуровневый доступ,
- **struct stat** — метаданные,
- **DIR / dirent** — каталоги,
- **passwd** — пользователи.

Эти примеры можно использовать как шаблоны.









